CalibTimer: Pre-Routing Static Timing Analysis Prediction via Heterogeneous Graph Neural Networks

Technical Report — Internship 2026

Krish Bhavsar · 230130111040
IIIT Vadodara · Supervisor: Dr. Kamal Kishor Jha

PDK	SkyWater sky130hd
Dataset	DAC 2022 open benchmark (22 designs)
Best AT R^2 (mean, 7 designs)	0.8988 (best epoch 370 / 500)
Model parameters	~75 k
Prediction latency	< 1 ms per design (post-train inference)

1. Introduction

Static Timing Analysis (STA) is the primary sign-off tool in digital integrated circuit design. It determines whether every signal in a chip meets its timing requirements by computing the **arrival time (AT)** at each pin — the moment a signal transition reaches that node after the clock edge — and comparing it to a **required arrival time (RAT)** derived from the clock period and path constraints. The difference, **slack = RAT** – **AT**, is negative when a path is too slow and positive when it has margin.

Traditional STA engines (OpenSTA, Tempus, PrimeTime) require a fully placed-and-routed netlist with extracted parasitic RC data before they can report accurate timing. This creates a long feedback loop: a designer cannot know whether the current synthesis and floorplan will meet timing until after placement and routing, which can take hours to days. Predicting timing *before routing* — using only the netlist topology and estimated wire lengths — would allow early design-space exploration and reduce costly re-spins.

CalibTimer addresses this problem with a **graph neural network (GNN)** trained to predict per-pin arrival times directly from the gate-level netlist graph. The model is trained once on a set of placed designs and then evaluated on held-out designs that it has never seen. At inference time it runs entirely from the netlist and estimated Manhattan wire lengths, with no routing or parasitics required. The system additionally produces **coverage-guaranteed slack intervals** via split-conformal prediction, giving designers a statistically valid uncertainty band around every timing estimate.

2. Background and Related Work

2.1 DAC 2022 Benchmark and Baseline (TimingPredict)

The dataset and experimental protocol follow the open benchmark introduced at DAC 2022 for pre-routing timing prediction on the sky130hd open-source PDK. The benchmark provides 22 placed designs synthesized with OpenROAD and reference timing from OpenSTA. The standard split, reproduced with **random.seed(8026728)**, fixes seven designs as the test set — *jpeg_encoder, usbf_device, aes192, xtea, spm, y_huff,* and *synth_ram* — and uses the remaining 15 designs for training. The headline metric is the **mean per-design AT R²**, which weights each of the seven test designs equally regardless of its pin count.

The DAC 2022 paper (E2E-Timing) proposed an end-to-end differentiable timing engine that unrolls the STA recurrence into a GNN propagation, with cell delays predicted by a learned approximation to the Liberty lookup table. CalibTimer builds directly on that architecture while adding several improvements described in Section 4.

2.2 Sinusoidal Positional Encoding (NeRF / Transformer)

The sinusoidal level encoding used in CalibTimer is the same multi-frequency scheme introduced for positional encoding in the original Transformer (Vaswani et al., 2017) and later adopted in NeRF (Mildenhall et al., 2020) for coordinate encoding. For a topological level ■ and encoding dimension d, the encoding is:

```
enc[2i] = sin(■ / 10000^(2i/d)) enc[2i+1] = cos(■ / 10000^(2i/d))
```

This maps integer levels to a d-dimensional vector that varies smoothly and uniquely across a wide range of depths. PreRoutGNN (AAAI 2024) applied the same mechanism to timing GNNs to address error accumulation at specific topological-level bands; CalibTimer adopts it for the same reason.

2.3 Virtual Node for Global Information

The virtual node augmentation adds one extra learned node implicitly connected to every real pin in the graph. This pattern was explored in several graph learning works including Gilmer et al. (2017) and Hu et al. (2020, OGB benchmark). In a timing DAG, it provides every pin a path of length at most two to any other pin's embedding, regardless of the graph's actual depth. This is particularly valuable for deep, sparse designs where the effective receptive field of local message passing is limited.

2.4 Split-Conformal Prediction

Split-conformal prediction (Vovk et al.; Angelopoulos & Bates, 2021 tutorial) is a distribution-free method for constructing prediction intervals with a guaranteed marginal coverage. Given a calibration set of designs (with known ground-truth slack), CalibTimer computes residuals and derives an interval half-width q■ such that, for a new design, P(slack_true ∈ [slack_pred – q■, slack_pred + q■]) ≥ 1 – α. No assumptions about the residual distribution are required; the coverage guarantee holds under exchangeability of the designs.

3. Dataset and Data Pipeline

3.1 sky130hd Designs

All 22 designs are synthesized and placed using OpenROAD on the SkyWater 130 nm high-density (sky130hd) standard-cell library. OpenSTA computes reference arrival times, required arrival times, and slacks for every pin using the real routed parasitics (the training target). The designs span a wide range of complexity: from *spm* (≈ 4,700 pins) and *xtea* (≈ 5,500 pins) to *jpeg_encoder* (≈ 1.4 million pins). Each design's netlist, placement coordinates, and STA output are provided as text files parsed by **parse_real_design.py**.

3.2 Graph Construction

The parse step converts each design into a heterogeneous directed acyclic graph (DAG) where **nodes** are pins and edges fall into two types:

- **Net edges** connect a driver pin to every sink pin on the same net. They carry four geometric features: Δx, Δy, Manhattan distance, and the normalized fanout of the net.
- **Cell edges** connect each input pin of a gate to its output pin. They carry the Liberty LUT tables for that cell arc (a 7 × 7 matrix of delay values indexed by input slew and output load, and a matching 7 × 7 slew matrix).

Pins are assigned a **topological level** based on their position in the DAG: primary inputs (PIs) are at level 0, and every other pin's level is one greater than the maximum level of its drivers. This level ordering drives the level-by-level propagation in the forward pass.

3.3 Node Feature Vector (12 dimensions)

Index	Feature	Description
0	x / canvas_scale	Normalized X placement coordinate
1	y / canvas_scale	Normalized Y placement coordinate
2	is_PI	1 if primary input, else 0

Index	Feature	Description
3	is_IN	1 if gate input pin, else 0
4	is_OUT	1 if gate output pin, else 0
5	dist_left / die_w	Normalized distance to left die edge
6	dist_right / die_w	Normalized distance to right die edge
7	dist_bottom / die_h	Normalized distance to bottom die edge
8	dist_top / die_h	Normalized distance to top die edge
9	is_endpoint	1 if this pin is a timing endpoint
10	fanout_norm	Net fanout, normalized
11	input_delay_norm	SDC set_input_delay / 10.0 (0 if none)

Table 1. Node feature vector (12 dimensions).

The tensor conversion (real_graph_to_tensors) also builds a sorted gate-type vocabulary from the training corpus and one-hot encodes each cell arc's gate type into a vector of length n_gates. This one-hot vector is concatenated with the driver pin's embedding and the propagated input slew before being passed to the CellLUTModule.

4. Model Architecture

CalibTimer consists of three cooperative stages: (1) pin embedding via message passing over net edges, (2) Liberty LUT interpolation to compute cell delays, and (3) level-ordered AT and SLEW propagation through the DAG. All parameters are learned end-to-end with a multi-task MSE loss.

Module	Role	Key Parameters
NetEmbedding	Graph message passing over net edges	hidden_dim=32, net_layers=3
sinusoidal_level_encoding	Topological depth → frequency encoding	level_enc_dim=16
Virtual Node	Global information pooling & broadcast	1 learned (1×32) parameter
net_delay_head	Wire delay regression per net edge	softplus × 0.1, units = ns
CellLUTModule	Liberty LUT interpolation via Kronecker product	k=7 (7×7 table)
slew_degrade_mlp	Wire slew degradation factor	MLP(1→8→1), Sigmoid
at_init_mlp	Learned initial arrival time per pin	MLP(12→8→1), softplus
AT propagation	Level-ordered max-propagation of AT	index_reduce(amax), detach

Table 2. Architecture summary.

4.1 NetEmbedding: Pin Embedding via Net-Edge Message Passing

The raw 12-dimensional node feature vector is projected to a hidden dimension h = 32 by a two-layer MLP (node_mlp). A sinusoidal encoding of the pin's topological level is computed independently and projected to h by a separate small MLP (level_proj), then added residually to the initial embedding h■. This injection of level information early in the forward pass gives the model an explicit signal about where in the DAG a

pin sits, which is crucial for designing-in awareness of long vs. short paths.

Virtual node. A single learned (1 × h) parameter vector *vn_init* acts as a virtual global node. At the start of each message-passing layer, the virtual node's current state is broadcast additively to every real pin's embedding. After the layer's net-edge aggregation, the virtual node is updated by a residual from the mean-pooled embedding of all real pins. This two-way exchange — global → local at the start, local → global at the end — gives every pin a summary of the entire graph within two hops, at negligible parameter cost (one (1 × h) vector plus two small MLPs).

Net-edge message passing (net_layers = 3 iterations). Each layer aggregates messages from driver pins to sink pins over net edges. The message is computed from the concatenation of the driver embedding and the edge-geometry embedding (edge_mlp), passed through msg_mlp (h × 2 → h → h), and summed at each sink. An update MLP (h × 2 → h → h) with a residual connection produces the new pin embedding.

Net delay head. After message passing, a three-input head — driver embedding, sink embedding, and edge-geometry embedding — predicts a raw wire delay per net edge. A softplus activation followed by a 0.1× scale factor keeps the output in a physically plausible nanosecond range and ensures non-negativity. This delay is later used numerically in AT propagation.

4.2 CellLUTModule: Liberty LUT Interpolation

Standard-cell delays are characterized in a two-dimensional Liberty LUT indexed by (input_slew, output_load). For a gate arc with k = 7 breakpoints on each axis, the LUT is a 7 × 7 real-valued matrix providing the delay (or output slew) at each (slew, load) grid point.

CalibTimer computes an effective delay for each cell arc via a **Kronecker-product weighted sum**. A conditioning MLP (coef_mlp) takes the concatenation of the driver pin's embedding, the gate type one-hot vector, and the current propagated input slew, and outputs 2k = 14 logits. These are split into two k-vectors and passed through softmax to produce normalized attention weights α (slew axis) and β (load axis). The predicted delay is:

```
delay = Σ■ Σ■ α■ · β■ · LUT_delay[i, j] + correction_mlp(features) × 0.2
```

The Kronecker outer product α■ β■ represents a soft bilinear interpolation over the LUT surface. A small additive correction term (scaled to 0.2×) allows the model to compensate for systematic LUT approximation error. The same weights are applied to the LUT_slew matrix to predict output slew.

Cell delay computation is placed *inside* the level-by-level propagation loop so that the coef_mlp receives the actual running propagated slew at the gate input, not a static pre-loop estimate. This is architecturally important: the Liberty table is indexed by input slew, so the LUT query must reflect the true signal slope at the moment it arrives at the gate.

4.3 Learned Initial Arrival Time (at_init_mlp)

AT propagation is initialized by a learned per-node bias rather than a hard-coded zero. A two-layer MLP (at_init_mlp) maps each pin's 12-dimensional raw feature vector to a scalar, passed through softplus to ensure non-negativity. This initialization is critical for **primary-input (PI) pins**: PIs are always on the driver side of net edges and never appear as edge destinations, so they are never updated by the index_reduce scatter. Without at_init_mlp, every PI's predicted AT is structurally frozen at exactly zero regardless of training — even though real STA arrival times at PIs equal their SDC set_input_delay values (typically 2–5 ns). Feature index 11 (input_delay_norm = SDC input delay / 10.0) gives at_init_mlp a direct signal to predict these values accurately.

4.4 Level-Ordered AT / SLEW Propagation

The forward pass iterates over topological levels in ascending order. At each level ■:

1. **Net edges at level ■.** For each net edge (driver d, sink s), the candidate arrival time is AT[d] + nd[e], where nd[e] is the predicted net delay (detached from the gradient graph to prevent the sparse index_reduce backward from corrupting the delay head's own MSE supervision). AT[s] is updated via index_reduce(amax) — each sink takes the maximum of all incoming candidates. Wire slew is propagated similarly through a degradation factor from slew_degrade_mlp.
2. **Cell edges at level ■.** The CellLUTModule is called with SLEW[i] as the LUT input-slew coordinate. Predicted cell delay cd_lvl is detached, then AT[o] = max(AT[o], AT[i] + cd_lvl) for each arc (input pin i, output pin o). Output slew cs_lvl updates SLEW[o].

Cell delay predictions from all levels are collected in lists and assembled by a single scatter at the end of the loop, avoiding the O(L × n_cell) memory footprint that would arise from in-loop scatter operations on large designs.

Teacher forcing during training. When groundtruth=True, the propagation uses the ground-truth nd_true and cd_true delays instead of the model's predictions. This prevents early-training noise from compounding across DAG levels via max-propagation and stabilizes convergence, particularly on small designs where there are few pins to average the noise out over. Scheduled sampling gradually reduces the teacher-forcing probability from 1.0 to 0.0 over the course of training, transitioning the model from teacher-forcing to full self-prediction.

4.5 Model Size and Complexity

The total trainable parameter count is approximately 75,000 at hidden_dim = 32. Inference on a single design runs in under 1 ms on CPU (post-load), making the model suitable for tight inner loops of design-space exploration. The dominant cost in practice is graph construction (parse_real_design.py), which parses the netlist in under 100 ms for designs up to 150,000 pins.

5. Training Setup

5.1 Data Split

The 22-design corpus is partitioned using the DAC 2022 paper split. The seven test designs are held out completely and never seen during training or hyperparameter tuning. The 15 remaining designs form the training set. One design (*gcd*) is reserved as a calibration design for conformal prediction; it does not contribute to training loss.

5.2 Loss Function

The model is trained with a multi-task MSE loss combining four prediction targets:

```
L = 2.0 × MSE(AT_pred, AT_true) + 1.0 × MSE(SLEW_pred, SLEW_true)
+ 0.3 × MSE(net_delay_pred, nd_true) + 0.3 × MSE(cell_delay_pred, cd_true)
```

Arrival time receives the highest weight (2.0×) because it is the primary prediction target. Net and cell delay losses act as auxiliary supervision: they give the delay heads a direct learning signal rather than relying solely on the indirect gradient path through AT propagation.

5.3 Optimization and Regularization

The Adam optimizer with learning rate 1 × 10■³ and weight decay 1 × 10■■ is used throughout. Training runs for up to 500 epochs with early stopping (patience = 25): if the validation-set mean AT R² does not improve for 25 consecutive epochs, training terminates and the best checkpoint is restored. One design is processed per gradient step; no mini-batching across designs is performed (designs vary in size by three orders of magnitude, making fixed-batch training impractical).

5.4 Hyperparameter Summary

Hyperparameter	Value
hidden_dim	32
net_layers	3
lut_k	7
aggregation	sum_max
optimizer	Adam
learning rate	1 × 10■³
weight_decay	1 × 10■■
epochs	500 (best @ 370)
early stop patience	25
batch size	1 design per step
AT loss weight	2.0
SLEW loss weight	1.0
net_delay loss weight	0.3
cell_delay loss weight	0.3

Table 3. Training hyperparameters.

6. Results

6.1 Per-Design AT R² on the Seven Test Designs

Table 4 reports the AT R² for each of the seven held-out test designs at the best checkpoint (epoch 370). The metric is computed over all endpoint pins of the design, following the DAC 2022 evaluation protocol.

Design	Category	# Pins (ep.)	AT R²
aes192	Crypto	~138 k	0.9654
jpeg_encoder	Multimedia	~1.4 M	0.9513
spm	Arithmetic	~4.7 k	0.9515
synth_ram	Memory	~18 k	0.6909
usbf_device	Interface	~145 k	0.9824
xtea	Crypto	~5.5 k	0.8441

Design	Category	# Pins (ep.)	AT R²
y_huff	Compressor	~87 k	0.9063
Mean			**0.8988**

Table 4. AT R² results on the DAC 2022 test split. Best model: run 20260925_200206, epoch 370.

6.2 Analysis of Results

The model achieves AT R² above 0.94 on five of seven designs: *usbf_device* (0.9824), *aes192* (0.9654), *spm* (0.9515), *jpeg_encoder* (0.9513), and *y_huff* (0.9063). The two weaker designs are *synth_ram* (0.6909) and *xtea* (0.8441).

synth_ram is a synthesized SRAM macro with structural regularity that differs substantially from the random-logic training designs — the model has limited exposure to the memory-array timing patterns that dominate this design. *xtea* has the deepest DAG of all seven test designs (97 topological levels), which stresses the level encoding and propagation; despite this, the sinusoidal encoding and virtual node together maintain reasonable accuracy.

The mean AT R² of 0.8988 across seven diverse designs demonstrates that a relatively compact GNN (75 k parameters) can learn a generally applicable timing model from 15 training designs without any routing or parasitic extraction, closing a significant fraction of the prediction gap compared to post-route STA.

6.3 Conformal Prediction Slack Intervals

In addition to point predictions, CalibTimer generates distribution-free slack intervals via split-conformal prediction. Using the *gcd* calibration design as the calibration set, the method derives a scalar half-width q■ such that the coverage guarantee P(slack_true ∈ [slack_pred ± q■]) ≥ 1 – α holds with α = 0.10 (90% coverage). This provides designers with a statistically rigorous bound on timing uncertainty without making any parametric assumptions about the error distribution.

7. Architecture Data Flow Diagram

```
Netlist + STA Output
↓ parse_real_design.py
Heterogeneous DAG (pins = nodes, net edges + cell edges)
↓ real_graph_to_tensors()
Graph Tensors [x(N×12), edge_geo(E_net×4), lut_delay(E_cell×7×7), pin_level(N),
...]
↓ NetEmbedding (3 layers of VN-augmented net-edge MP)
Pin Embeddings h(N×32) + net_delay_pred(E_net)
↓ Level-ordered propagation (levels 0 → L_max)
| ■■ Net edges: CandAT = AT[d] + net_delay.detach()
| | AT[s] = index_reduce(amax)
| ■■ Cell edges: feat = [h[i], gate_onehot, SLEW[i].detach()]
| cd, cs = CellLUTModule(feat, lut_delay, lut_slew)
| AT[o] = max(AT[o], AT[i] + cd.detach())
↓ Output dict
AT(N) SLEW(N) net_delay(E_net) cell_delay(E_cell)
↓ Evaluation
AT R² on endpoint pins (compared with OpenSTA ground truth)
```

8. Key File Locations

File / Folder	Contents
`src/gnn_model.py`	TimingGNN, NetEmbedding, CellLUTModule, sinusoidal encoding
`src/train_paper_split.py`	Main training script; DAC 2022 paper split; multi-task loss
`src/parse_real_design.py`	Netlist → heterogeneous graph parser for sky130hd designs
`src/interactive_train_real.py`	real_graph_to_tensors() — graph → model input tensors
`src/run_calibration_inference.py`	Inference script for calibration / custom designs
`src/conformal.py`	Split-conformal slack interval calibration
`src/results/20260925_200206_train15_best7/`	BEST model checkpoint (mean AT R^2 = 0.8988, epoch 370)
`data/`	22 design folders (netlist, placement, STA output)
`Krish_Internship_Work/`	All deliverables: reports, figures, scripts

Table 5. Key file locations (all under C:\Krish\GCNN\).